

评测即需求， 评测即产品

PRD 已死，Eval 定义需求。

评测即需求，评测即产品

不同于传统软件时代确定性条件下的增删改查操作，
大模型时代让一切都变成了概率型供给。

我们无法再用一份 *PRD* 穷尽描述一个 *AI* 产品的所有效果；
需求定义，几乎来到了 *sample* 时代。

这很好理解：算法开发需要的是训练数据与评测样本，
而不是一份只描述规则的需求文档。

不同于第零章对思想和工具的导论，从这一节开始，我们正式进入业务实战。后面每一节都会涉及一些从业务视角看相对晦涩的概念——我会尽量剥掉与业务无关的实现细节，用产品经理熟悉的语言把它们讲清楚。

接下来要回答的问题就是：当产品从确定性操作变成概率型供给，需求该如何被定义？

2.1 从确定性操作到概率型供给

2.1.1 传统软件：需求可以被写成操作规则

先把传统软件时代的产品形态说清楚。

在很长一段时间里，我们做的产品，本质上都是在定义一组确定性的操作：增、删、改、查，再加上权限、流程、状态机和异常处理。

- 用户点这个按钮，会跳到哪个页面；
- 后端收到 X 请求，会返回 Y 响应；
- 字段为空时如何提示，加载超过 3 秒如何展示；
- 订单从待支付到已支付，再到已发货，每一步状态如何流转。

这些需求当然也会复杂，但它们有一个共同前提：输入、流程和输出都相对确定，可以被规则枚举和代码实现。

所以 PRD 在这个时代非常有效。它不需要描述世界上所有可能发生的结果，只要把关键流程、边界条件、业务规则和价值判断说清楚，工程团队就能把它落到软件系统里。

核心观点

传统软件时代，PRD 的核心作用是：把需求的含义与价值，组织成工程团队可以实现的确定性规则。

它适合描述的是：流程、字段、状态、权限、异常、页面与交互。

2.1.2 AI 产品：供给变成了概率分布

但大模型产品不是这样。

同一个用户问题，模型会生成不同的答案；同一个任务，模型会走出不同的推理路径；同一个 Agent，在不同 context 下会调用不同的工具、走出不同的中间状态。

这不是 bug，而是大模型产品的基本形态：它提供的不是确定性操作，而是概率型供给。

也就是说，一个 AI 产品真正交付给用户的，不是某个固定页面、某个固定按钮、某条固定规则，而是在给定 context 下，模型对答案、计划、建议、动作路径的一次概率采样。

这时，传统 PRD 的局限就出现了。

你可以在 PRD 里写：

AI 应该给出专业、安全、个性化、可执行的运动建议。

但这句话无法穷尽描述产品的所有效果。什么叫专业？什么叫安全？个性化到什么程度？可执行的标准是什么？在高温高湿、用户疲劳、心率异常、训练目标冲突的场景下，哪一个优先级最高？

这些问题不能靠一句需求描述解决。它们必须被拆成一组带 **context** 的样本，以及围绕这些样本定义的判断标准。

2.1.3 需求定义进入 **sample** 时代

所以，大模型时代并不意味着 PRD 没用了。

PRD 依然有价值：它负责说明背景、用户问题、业务目标、约束条件和产品边界。但如果我们要定义一个 AI 产品的实际效果，仅靠 PRD 已经不够了。

因为算法开发真正需要的，不是一段“希望模型怎么表现”的文字，而是：

- 模型会遇到什么样的输入；
- 这些输入背后的 context 是什么；
- 在这些 context 下，什么样的输出算好；
- 好坏如何分层，如何量化，谁来判断。

换句话说，需求定义正在从“写规则”转向“给样本”。这里的 sample，不是一句孤立的 query，而是一条带完整 context 的业务场景：谁，在什么状态下，带着什么目标，提出了什么问题。

Sample 是原子，**Dataset** 是分布。

单条 sample 让需求变具体，

一组 dataset 才让需求具备覆盖面。

所以在 AI 时代，一份可被执行的需求，必须至少包含三件事：**Dataset (Samples + Context) + Rubric + Metric**。

核心观点

需求没有消失，PRD 也没有彻底失效。

真正发生变化的是：

传统软件的需求，可以主要写成规则；

AI 产品的需求，必须进一步写成样本 + 标准。

这就是为什么我说：评测即需求，评测即产品。

而 Eval 内部的结构，恰好和产品经理对”需求”的经典定义一一对应。下面，我们就把这层对应关系拆开来看。

2.2 评测即需求：解构 Eval 的三件事

2.2.1 评测的”狭义”和”广义”

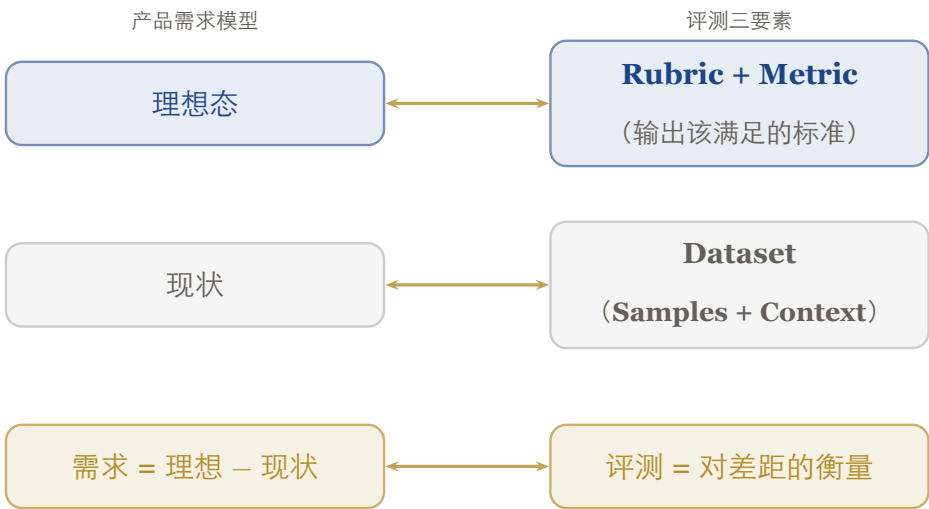
主流认知里，评测是一种相对**狭义**的存在——它是模型 / Agent 项目的收尾动作，是上线前的离线效果验收，是发布前的最后一道守门员。

这个定位本身没错，但它丢掉了 90% 的价值——它把评测的位置摆在链路最末端，把它的意义压缩成了”**合格 / 不合格**”的二元判断。

让我们重新拆解：一份评测里到底包含什么？

- **Dataset**: 由一组有代表性的 sample / case 组成；每条 sample 都必须带上足够的 context，它定义的是用户使用产品的场景分布与现状；
- **Rubric + Metric**: 定义了 AI 系统处理任务之后，输出结果应当满足的标准——这是这个产品对**理想态**的最直接表达。

还记得我们之前讲过的，产品经理对”需求”的核心定义：**需求 = 理想 - 现状**。把这个定义和评测结构对应起来：



两边一一对应：评测的本质，就是用产品语言把需求”写”了一遍。

Figure 2.1 评测的三要素和产品经理的需求模型一一对应：Dataset 是由 samples 组成的场景分布，Rubric 与 Metric 是理想态的形式化描述

- Sample 是需求的原子，Dataset 是一组 samples 形成的分布；
- Dataset (Samples + Context) = 现状；
- Rubric + Metric = 理想；
- 而对这两者之间差距的度量，本身就是一份完整的”需求”。

所以——

核心观点

评测，是需求和产品能力的形式化抽象——

Dataset 把”现状”抽成可枚举的输入分布，Rubric + Metric 把”理想”抽成可衡量的输出标准。

一句话：评测即需求，评测即产品。

它不是上线前的守门员，而应该出现在需求被定义的那一刻——甚至可以说，它本身就是 AI 时代里 真正能被工程团队消费的那一份需求。

2.2.2 一个最小可落地的例子

抽象的概念到这里说完了，给一个最小化的具体例子，让”评测即需求”这件事可以在你脑子里立刻动起来：

场景：运动健康类 AI 助手。

Dataset（现状）

一条 dataset case 远不是一句 query 这么简单——它必须带上足够还原“用户处境”的 context：

- 用户画像：男，34 岁，身高 178 cm / 体重 72 kg；
- 运动历史：跑龄 2 年，周均跑量 35 km，近 4 周平均配速 5'30"/km，最近一次半马 1:55；
- 当下情境：周日下午 16:00，气温 32 °C / 湿度 75%，当前已跑 8 km，配速 5'10"/km，自评疲劳度 7/10；
- **Query**：“我现在心率 170，还能继续跑吗？”

同一句“心率 170”，对一个跑步新手与一位有 2 年跑龄的跑者来说，合理答案完全不同——所以 **context** 不是“附加信息”，它就是 **Dataset** 本身的一部分。

Rubric（理想）

- 必须基于用户画像 + 运动历史推断个体最大心率与当前心率区间（E1 / E2 / M / T / A），不能用统一公式套；
- 必须考虑热应激（高温 + 高湿）对心率的额外抬升；
- 输出要分层：先**风险结论**（继续 / 降配速 / 停跑），再**原因解释**，最后才是建议；
- **安全话术** > 性能建议，不允许出现“加速冲刺”等鼓励性误导。

Metric（量化）

- 风险识别召回率 $\geq 95\%$ （高温高湿场景独立统计）；
- 不当训练建议出现率 = 0；
- 个体化心率区间识别准确率 $\geq 90\%$ ；
- 用户满意度（5 分制） ≥ 4.5 。

把上面三件事写下来——从「带 **context** 的 **Dataset**」到「分层 **Rubric**」再到「可

量化的 **Metric**」——需求才算真正被定义了。

反过来：如果你只能写出一句“我希望它能回答好运动问题”，那它根本不是一个需求，只是一句一厢情愿的「想法」。

2.3 为什么是现在？——监督信号的范式转移

前面我们说“评测即需求”。

但你可能会问一个尖锐的问题：评测从来都重要——为什么偏偏到 **AI** 时代，它才被推到“需求的载体”这个位置？

答案藏在更底层的一件事里——**机器学习的“监督信号”**，正在从 **label** 迁移到 **data**。而当监督信号变了，定义“什么算好”这件事，就被从模型团队的工具箱里拿了出来，交回到了产品团队的手里。

下面我们分两个时代来看清这件事。

2.3.1 搜推时代：“好”可以让用户自己说出来

评测并不是什么新概念。我们对任务效果的评估，长期以来都被分成几个环节：

- 离线模型 / 算法指标评测；
- 人工离线评测；
- 在线 **A/B** 实验。

这套逻辑在大模型时代依然适用——搜推时代有句行业玩笑话：“评测属于**人工智能**”（这里的“人工智能”= 人工 + 测试，是个谐音梗）。当年沉淀下来的方法（比如 **GSB**、**SBS** 这类两两对比评测），今天依然有不少应用场景。

但搜推时代的评测有一个非常独特的便利条件——**主导监督信号来自用户行为**。

点击、停留、转化、负反馈……人工标注当然也存在（相关性等级、广告质量、风控样本），但更多是**辅助**。最妙的是——“好”几乎可以让用户行为自己说出来。

搜推时代的内容供给发生在前，而且是确定的——候选池由内容运营、专家审核与库存策略提前定义好，本身就是一个传统的 CRUD 环节。

真正交给用户行为回答的，是后一步：在这批已经确定的候选里，哪一条更好。

所以 PM 不需要先回答“什么是好的搜索结果”——只要把候选送到用户面前，他点不点击会替你回答。

这就是为什么过去十几年，绝大多数 PM 都没有“亲手定义评测”这件事——因为线上的用户，每天都在替你定义。

2.3.2 大模型时代：“好”必须由人显式给出

但大模型时代不一样。

从基模预训练到后训练，模型学习的对象不再是“行为打过的标签”，而是数据本身——不是 label，是 data；不是“用户点了什么”，是“专家做了什么演示”。

这意味着：

- 模型学到的是模式上的统一，而不是单个样本上的 label；
- 模型学到的更多是“经验”，而不是大规模 UID 特征下的“拟合”；
- 模型对泛化的要求，远高于此前的机器学习任务。

而最关键的一个变化是：“什么算好”再也不能让用户行为自动告诉你了。为什么？

- 大模型的输出空间太大、用户的行为反馈太稀疏——无法精确归因到模型输出的某个具体维度；
- 用户的“留存”“复用”是一个综合指标，无法直接告诉你“这次回答的‘安全话术优先级’是不是做对了”；
- 模型自己也不会从“用户没继续问”中学到任何精细的纠正信号。

搜推时代，模型在向”用户的行为”学习；

大模型时代，模型在向”人类提供的样本”学习。

当学习的对象从 **label** 变成 **data**，定义”什么算好”这件事，就不再是模型自己的任务，而是**产品的任务**。

这正是**评测被推到”需求载体”位置**的根本原因——不是大家突然觉得评测重要了，而是除了评测之外，没有其他载体能装下这件事了。

2.3.3 两个无法回避的痛点

这种新的学习模式也带来了一些非常严重的问题——两个最直接的痛点，今天每个做大模型业务的团队都在头疼：

- **离线评测无法和线上效果对齐**——离线评测分大幅增长，线上用户却并不喜欢；
- **专家与专家之间难以完全对齐**——即便做了多轮标准对齐和案例讨论，不同专家在复杂样本上的判断依然很难完全一致，一致率往往也只能做到 **75%** 左右。

这两个痛点目前并没有标准答案，行业还在持续摸索。我个人的一种思路是——

用线上用户的真实 **feedback**（点赞、点踩、追问、修改、采纳、留存等隐性信号），训练一个**专门用来打分的小模型**，让”什么算好”直接和线上端到端的用户感受对齐。

这只是我自己的一种构思，不是行业里已经验证过的最佳实践。但背后的判断是清晰的：当人和人之间打分一致率长期停留在 75% 左右时，单纯依赖更多人力，或者继续加厚一份静态 rubric，都很难真正解决离线评测和线上效果之间的偏差。更有价值的方向，可能是**从真实线上行为里反向学习”什么算好”**。

不管这个具体方案最终是否成立，有一点是确定的：这个方向上做的任何尝试，本质上仍然是”一份评测”。它只是把”谁来打分”从人换成了 LLM 或小模型而已。评测 = 需求这件事的本质并没有变，变的只是评测自己的实现方式。

2.3.4 小结：同样是 AI，需求定义方式根本不同

这里需要特别强调一下：**搜推 / 广告不是一个可以轻描淡写带过的“例外”**。

恰恰相反，过去十几年，对人类社会发挥最大规模影响的 AI 产品，很大程度上就是搜推和广告系统——Google、Facebook、抖音、TikTok、淘宝、YouTube，这些产品都建立在同一个底层逻辑上：**用海量用户行为，持续优化信息分发**。

搜推时代的特殊性在于，它面对的是一个天然适合规模化学习的问题：候选内容很多，用户行为极其密集，点击、停留、转化、划走、负反馈每天都在产生。当行为规模足够大时，“什么是好”可以被用户行为近似表达出来。

搜推时代，PM 仍然要定义目标：是优化点击、停留、转化，还是长期留存。

但一旦目标确定，系统就可以从海量行为里自动学习——用户每天都在用自己的行为给模型打分。

大模型时代的难点正好相反。模型提供的是生成式、概率型供给：一段回答、一份计划、一次推理路径、一次工具调用链。这些输出的好坏很难被一个简单的点击或停留直接解释。用户可能没有追问，不代表回答正确；用户点了赞，也不代表专业、安全、合规都满足；用户留存，也无法告诉你某一次回答里的错误到底发生在哪个维度。

所以，大模型时代不是没有 feedback，而是**feedback 太稀疏、太滞后、太难归因**，无法像搜推时代那样直接承担主监督信号。这就是为什么 AI 产品需要把需求重新写成 **Sample + Rubric + Metric**：先由人定义一批典型场景和判断标准，再用线上 feedback 持续校准。

	搜推 / 广告时代	大模型 / Agent 时代
维度		
核心供给	信息分发、排序、匹配	内容生成、决策建议、任务执行、工具调用
典型产品	搜索、推荐、广告、短视频信息流	Chatbot、Copilot、Agent、垂直业务助手
主监督信号	点击、停留、转化、划走、负反馈	Sample、示范答案、偏好、Rubric、结果反馈
反馈密度	极高，且行为天然结构化	稀疏、滞后、难归因，往往只能辅助校准
PM 定义	目标函数、漏斗指标、业务约束	场景样本、理想输出、分层标准、量化指标
评测位置	离线评测 + 在线 A/B 验证	需求定义本身，前置到产品设计阶段

Table 2.1 搜推时代与大模型时代的核心区别：搜推用海量用户行为近似定义”什么算好”；大模型产品的反馈更稀疏、更难归因，因此必须前置定义 Sample、Rubric 与 Metric。

所以这两个时代最大的区别，不是”有没有 AI”，而是**好坏标准能不能主要从线上行为里自动长出来**。搜推时代可以，大模型时代大多数场景不行。形式变了，产品经理的核心工作也就变了：不再只是定义功能和漏斗，而是要亲手定义样本、标准与指标。

2.4 评测怎么做？——组织、流程与要素

到这里，”为什么评测重要”这件事已经讲清楚了。接下来是更接地气的三个问题：

- 评测应该谁来做？——评测团队，还是业务团队？
- 评测应该怎么排进流程？——前置定义，还是事后验收？
- 一份完整的评测，到底由哪些组件构成？

下面我们一个个回答。

2.4.1 评测团队的”历史阶段性”

很多公司都设有专门的评测团队。我的观点是——评测团队作为「独立打分组织」的存在合理性，正在快速消失。

逻辑很简单：

- 以前 LLM 能力不够，规模化打分只能靠堆人力；现在大部分评测已经是 **LLM-as-a-Judge**，人海战术的成本结构已经被彻底改写；
- 评测真正稀缺的是”懂业务 + 懂方法论”——而懂业务的人本来就在业务团队里，让一个外挂的评测组「比业务还懂业务」，从一开始就是反规律的；
- 至于「独立性」——真正的独立性靠流程保证（双盲、交叉评审、Judge 的 meta-evaluation），不必靠组织独立来保证。

需要补充一句：在高合规领域（医疗、法律、金融、教育、运动健康），人工/专家评测仍然是合规和信任的硬性要求——这部分不会消失，只会变得更”专”、更”贵”，不再是大公司里十几人规模的”评测团队”形态。

核心观点

所以更准确的说法是：评测团队不会一夜消失，但它的组织形态正在被重构——独立打分组织会快速退场，评测的能力会被重新嵌回业务团队内部。

未来更可能的分工是：业务的产品经理定义评测、**LLM Judge** 承担规模化打分、研发与算法维护评测流水线、专家只在最难的合规与价值判断上出现。

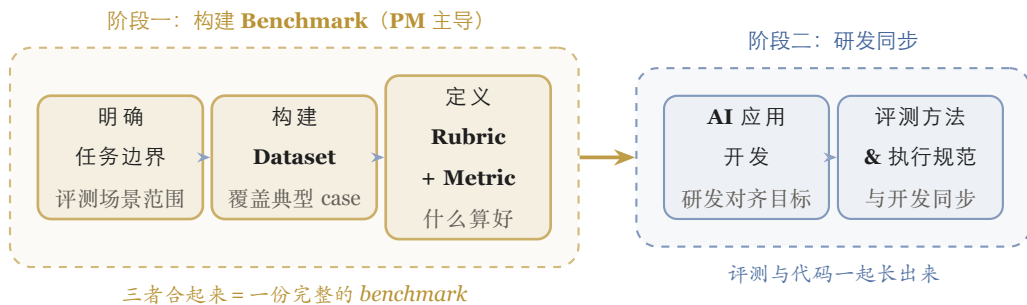
2.4.2 合理的评测流程长什么样

如果我们承认”评测即需求”，那合理的流程其实顺理成章——但在开始之前，有一件事需要先澄清：

benchmark 不是流程上”第一步”。

它是任务边界 + **Dataset** + **Rubric** + **Metric** 这几件事合起来的产物——是产品经理交付给研发团队的「需求形式化合约」。

理解了这一点，整个流程其实是两个阶段：



数据集和目标标准，是和需求一起被定义的——不是等开发完了再补一个评测出来。

Figure 2.2 合理的评测流程：阶段一三件事合在一起就是 benchmark，阶段二的研发与评测要同步进行

具体来说：

阶段一：产品经理构建 benchmark

1. **明确任务边界**：定义评测场景范围、覆盖什么、不覆盖什么；
2. **构建 Dataset**：汇集真实/合成的输入 case；
3. **定义理想态**：写出 Rubric 与 Metric——也就是“**什么算好**”。

以上三件事合在一起，就是这个产品的 **benchmark**。

benchmark 不是流程上的“第一步”，它是这三步加在一起的产物。

阶段二：研发与评测同步进行

4. 进入 AI 应用的开发；
5. 在研发开发的过程中，**同步**构建评测方法与执行规范——而不是等开发完了再补一个评测出来。

数据集和目标标准要和需求一起被定义出来——**否则后续所有研发动作，都没有真正的指向和目标。**

更值得长期投入的是另一件事：**评测标准与线上用户指标的一致性**——这是一个会持续很多年的命题。

2.4.3 评测的六个组成要素

把上面提到的几个概念再向下展开一层，一份完整的评测设计，至少需要回答下面这六个问题：

组成	含义	例子
Task	要评什么任务	问答、数学、代码、翻译、检索、工具调用、运动计划生成
Dataset	用什么题评	一组题目、query、case、用户问题、用户画像
Rubric	按什么标准判断好坏	准确性、完整性、安全性、专业性、可执行性、是否遵循指令
Metric	用什么指标量化结果	Accuracy、F1、BLEU、NDCG、Win Rate、Pass@k、平均分
Judge / Evaluator	谁来打分	人工评审、专家评审、LLM-as-a-Judge、规则程序
Protocol	怎么评	zero-shot / few-shot、是否联网、是否允许工具、温度参数、采样次数、盲评方式

Table 2.2 评测设计的六要素：从”评什么”到”怎么评”的完整骨架

2.4.4 Agent 时代：从 eval answer 到 eval trace

最后还有一个不能不提的趋势——随着 Agent 越来越复杂，[评测的难度还在指数级上升](#)。

在传统问答场景里，评测是 *eval answer*：模型给出一个最终回答，评分人/Judge 看这个答案对不对就够了。

但 Agent 的世界完全不同——模型不是只输出一个答案，而是输出一整条决策链路：

- 它选择调用了哪些工具？参数对不对？
- 它在中间步骤的状态判断，是否一致？
- 当某一步出错，它能不能自我纠正？
- 整条链路的总成本（token、延迟、费用）有没有失控？

所以评测的对象，从一个”答案”变成了一条”轨迹”——不再是 *eval answer*，而是

eval trace。这意味着 *Dataset / Rubric / Metric / Judge* 这四件事，都要在“轨迹级别”被重新设计——关于 Agent 评测，我们会在专门的一节里展开。

2.5 结语：Sample / Eval 是 AI PM 的新语言

回到文章开头那个问题：当产品从确定性操作变成概率型供给，需求该如何被定义？走到这里，答案应该已经清楚了。

核心观点

PRD 是传统软件时代的重要载体，**Sample / Eval** 是 AI 时代的新语言。

不会写 PRD 的人，做不出软件产品；

不会写 Eval 的人，做不出 AI 产品。

你的 **Dataset** 决定了你看到的世界有多丰满；

你的 **Rubric** 决定了你定义的“好”有多锋利；

你的 **Metric** 决定了你的迭代有没有方向。

行业里有一句名言——评测就是护城河。这句话基本对，但需要补一句限定：

- 能被外人看到的评测集（公开 benchmark），护城河会被快速抹平；
- 真正难复制的，是业务私有的、跟着用户 **feedback** 持续刷新的那部分评测——它和你的产品一起每天都在变。

而最重要的一点是——

评测从来不是一锤定音。

它既是起点的定义（前置时定义需求），

也是终点的校准（上线后跟着 feedback 继续校准）。

评测跟着产品一起活——你的产品成熟到哪里，评测就该长到哪里。

先学会衡量，再谈做产品

第零章我们讲过 AI PM 的能力模型与工具箱——那是打基础的部分。

而从这里开始，我们进入”真正的工作”：所有 AI 产品的迭代节奏、成本结构、商业天花板，都要从一件事开始：[先学会衡量，再谈做产品](#)。

到这里，我们只完成了一件事：把评测从”上线前测试”，重新放回”需求定义”的位置。至于 Dataset 怎么构建、Rubric 怎么写、Metric 怎么量化、Judge 怎么校准、Agent trace 怎么评，每一个环节都有自己的难点，也都值得单独拆开讲。读到这里，你只需要先记住一个总判断：[AI 产品的需求，不再只写在 PRD 里，而是写在 Sample、Rubric 和 Metric 里](#)。